

TEDx



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO



Panoramica



Navigazione tra Talk Suggestiti

- *Estendere "MyTEDx" con una funzionalità che permetta agli utenti di vedere talk suggeriti e capire perché quei talk sono suggeriti (analisi esplicativa).*
- *Integrare un modello di raccomandazione (es. basato su contenuto o collaborativo).*
- *Funzione Lambda in AWS che, data una caratteristica, restituisce una lista di talk consigliati.*



Nuova Funzionalità sulla Board

- *Aggiungere la trascrizione completa per ogni talk*
- *Elaborare riassunti e approfondimenti tramite l'API di OpenAI prendendo come ingresso la trascrizione del task corrente*
- *Implementazione della gestione della ricerca dato in input*

```

Variabili d'ambiente (da configurare nella Lambda)
NEO4J_URI = os.environ.get('NEO4J_URI')
NEO4J_USER = os.environ.get('NEO4J_USER')
NEO4J_PASSWORD = os.environ.get('NEO4J_PASSWORD')

driver = None

def get_neo4j_driver():
    global driver
    if driver is None:
        try:
            driver = GraphDatabase.driver(NEO4J_URI, auth=basic_auth(NEO4J_USER, NEO4J_PASSWORD))
            # Verifica la connessione (opzionale ma utile per il debug iniziale)
            driver.verify_connectivity()
            print("Successfully connected to Neo4j.")
        except Exception as e:
            print(f"Error connecting to Neo4j: {e}")
            # Se la connessione fallisce, driver rimarrà None e gestito dopo
            # Potresti voler sollevare un'eccezione qui se la connessione è critica all'avvio
            raise ConnectionError(f"Failed to connect to Neo4j: {e}") from e
    return driver

def get_connected_nodes(tx, node_id_param):
    query = (
        "MATCH (startNode {id: $node_id_param})--(connectedNode) "
        "RETURN connectedNode.title AS title, "
        "        connectedNode.speakers AS speakers, "
        "        connectedNode.description AS description"
    )
    result = tx.run(query, node_id_param=node_id_param)

    nodes_data = []
    for record in result:
        nodes_data.append({
            "title": record["title"],
            "speakers": record["speakers"], # Assumendo sia una lista o una stringa
            "description": record["description"]
        })
    return nodes_data

```

```

except ConnectionError as ce:
    print(f"Connection Error: {ce}")
    return {
        'statusCode': 503, # Service Unavailable
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'error': 'Database connection failed', 'details': str(ce)})
    }
except Exception as e:
    print(f"Error processing request: {e}")
    # Includere str(e) può esporre dettagli, valuta per produzione
    return {
        'statusCode': 500,
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'error': 'Internal server error', 'details': str(e)})
    }

```

```

def lambda_handler(event, context):
    print(f"Received event: {event}")

    try:
        # Estrai 'id' dai parametri della query string (tipico per GET via API Gateway)
        query_params = event.get('queryStringParameters', {})
        if not query_params: # Prova a vedere se arriva nel body (per POST o test diretti)
            try:
                body = json.loads(event.get('body', '{}'))
                node_id = body.get('id')
            except json.JSONDecodeError:
                node_id = None
        else:
            node_id = query_params.get('id')

        if not node_id:
            return {
                'statusCode': 400,
                'headers': {
                    'Content-Type': 'application/json',
                    'Access-Control-Allow-Origin': '*' # Per CORS, adattare se necessario
                },
                'body': json.dumps({'error': 'Parameter "id" is missing'})
            }

        print(f"Querying for connections to node with id: {node_id}")

        # Ottieni il driver
        db_driver = get_neo4j_driver()

        connected_nodes_list = []
        with db_driver.session() as session:
            connected_nodes_list = session.read_transaction(get_connected_nodes, node_id)

        print(f"Found {len(connected_nodes_list)} connected nodes.")

        return {
            'statusCode': 200,
            'headers': {
                'Content-Type': 'application/json',
                'Access-Control-Allow-Origin': '*' # Importante per le app mobili/web
            },
            'body': json.dumps(connected_nodes_list)
        }

```



```
# --- Neo4j Query Functions ---
def create_or_update_talk_node(tx, talk_data):
    """
    Uses MERGE to create a Talk node or update properties.
    Uses 'id' property derived from MongoDB 'id'.
    All other fields from MongoDB (except 'next_watch') are set as properties.
    """
    talk_id = str(talk_data['_id']) # Assicura che sia stringa

    props = {}
    for k, v in talk_data.items():
        if k not in ['_id', 'next_watch'] and v is not None: # Esclude _id, next_watch e valori null
            if isinstance(v, list):
                # Assicura che le liste contengano solo tipi primitivi supportati da Neo4j
                # o che il driver Python possa convertire (es. str, int, float, bool).
                # Per tipi più complessi in liste, sarebbe necessaria una gestione ad-hoc.
                props[k] = [item for item in v if isinstance(item, (str, int, float, bool))]
            elif isinstance(v, (str, int, float, bool, dict)): # Neo4j supporta mappe (dict)
                props[k] = v
            # Altri tipi (es. oggetti non serializzabili) verrebbero ignorati.
            # Se ci sono tipi specifici da gestire (es. ObjectId di Mongo non stringhizzati),
            # andrebbero trattati qui.

    # --- Gestione Tipi Specifica ---
    # Duration: converti in intero se è una stringa numerica
    if 'duration' in props and isinstance(props['duration'], str):
        try:
            if props['duration'].isdigit():
                props['duration'] = int(props['duration'])
            else:
                # Se non è un intero valido, potrebbe essere meglio rimuoverlo o loggare un errore più specifico
                print(f"Warning: duration '{props['duration']}' for talk {talk_id} is not a simple numeric string. Storing as string or consider specific handling.")
        except ValueError: # Mantenuto per sicurezza, anche se isdigit() dovrebbe coprire
            print(f"Warning: Could not convert duration '{props['duration']}' to int for talk {talk_id}. Storing as string.")
    elif 'duration' in props and not isinstance(props['duration'], (int, float)):
        # Se duration è presente ma non è né stringa né numero (improbabile data la logica sopra)
        print(f"Warning: duration for talk {talk_id} has an unexpected type: {type(props['duration'])}. Removing property.")
        del props['duration']
```

```
        if related_id is None or related_id.strip() == "" or str(source_id) == related_id:
            if str(source_id) == related_id:
                pass # Skip self-relationship
            else:
                print(f"Skipping invalid/null related_id '{related_id}' for source {source_id}")
                continue

        created_relationships_attempts += 1
        try:
            session.execute_write(create_relationship, source_id, related_id)
            if created_relationships_attempts % 100 == 0:
                print(f"Attempted to create/merge {created_relationships_attempts} relationships...")
        except Exception as e:
            # create_relationship gestisce errori di query, questo catch è per errori di transazione
            print(f"Unexpected error during relationship write transaction ({source_id})->({related_id}): {e}")
            traceback.print_exc()
            continue

    print(f"Finished Phase 2. Attempted to create/merge approximately {created_relationships_attempts} relationships (MERGE skips duplicates).")
    print("Data synchronization process completed.")

except Exception as e:
    print(f"Error fetching data from MongoDB or during processing phases: {e}")
    traceback.print_exc()
    sys.exit("Job failed during data fetching or processing.")

except (pymongo.errors.ConnectionFailure, neo4j_exceptions.ServiceUnavailable, neo4j_exceptions.AuthError) as e:
    print(f"FATAL: Database connection or authentication error during initialization: {e}")
    traceback.print_exc()
    sys.exit("Job failed due to database connection/auth error.")

except Exception as e:
    print(f"FATAL: An unexpected error occurred: {e}")
    traceback.print_exc()
    sys.exit("Job failed due to an unexpected error.")

finally:
    if clients_initialized_successfully:
        print("Executing final client cleanup...")
        cleanup_clients()
    else:
        print("Skipping client cleanup as initialization may have failed.")
    print("AWS Glue Python Shell Job Finished.")
```

```
# publishedAt: converti in oggetto datetime se è una stringa ISO 8601
# Il driver Python per Neo4j convertirà l'oggetto datetime di Python
# nel tipo DateTime di Neo4j.
if 'publishedAt' in props and isinstance(props['publishedAt'], str):
    try:
        iso_string = props['publishedAt']
        # Python 3.7+ datetime.fromisoformat gestisce 'Z' (UTC Zulu).
        # Per versioni precedenti o per maggiore robustezza con formati leggermente diversi:
        if iso_string.endswith('Z'):
            # Sostituisce 'Z' con '+00:00' che fromisoformat comprende universalmente per UTC
            dt_obj = datetime.fromisoformat(iso_string[:-1] + '+00:00')
        else:
            # Prova a parsare direttamente, sperando sia un formato ISO compatibile
            dt_obj = datetime.fromisoformat(iso_string)
        props['publishedAt'] = dt_obj
    except ValueError:
        print(f"Warning: Could not parse publishedAt string '{props['publishedAt']}' into datetime for talk {talk_id}. It will be stored as a string if possible, or skipped.")
        # Lascia props['publishedAt'] come stringa se il parsing fallisce. Neo4j lo memorizzerà come stringa.
        # Se si preferisce non memorizzare stringhe malformate, si può fare: del props['publishedAt']

query = (
    "MERGE (t:Talk {id: $talk_id}) "
    "ON CREATE SET t = $props, t.id = $talk_id " # t.id è ridondante se props contiene id, ma sicuro
    "ON MATCH SET t += $props, t.id = $talk_id " # Sovrascrive/aggiunge proprietà, t.id assicura che non venga perso
)
tx.run(query, talk_id=talk_id, props=props)
```



```
print("Fetching talks data from MongoDB...")
try:
    talks_cursor = collection.find({})
    talks_data = list(talks_cursor)
    found_count = len(talks_data)
    print(f"Found {found_count} talks in MongoDB.")

    if not talks_data:
        print("No data found in MongoDB collection. Job will exit successfully.")
    else:
        print("Phase 1: Creating/Updating Talk nodes in Neo4j...")
        with neo4j_driver.session(database="neo4j") as session:
            for i, talk in enumerate(talks_data):
                if 'id' not in talk or talk['id'] is None:
                    print(f"Skipping document at index {i} due to missing or null 'id'.")
                    continue
                try:
                    session.execute_write(create_or_update_talk_node, talk)
                    processed_nodes += 1
                    if processed_nodes % 100 == 0 or processed_nodes == found_count:
                        print(f"Processed {processed_nodes}/{found_count} nodes...")
                except Exception as e:
                    print(f"Error processing node for talk ID {talk.get('id', 'N/A')}: {e}")
                    traceback.print_exc()
                    continue
            print(f"Finished Phase 1. Processed {processed_nodes} nodes.")

        print("Phase 2: Creating RELATED TO relationships in Neo4j...")
        with neo4j_driver.session(database="neo4j") as session:
            for i, talk in enumerate(talks_data):
                source_id = talk.get('id')
                next_watch_list = talk.get('next_watch')

                if not source_id:
                    print(f"Skipping relationship creation for talk at index {i} without source ID.")
                    continue

                if isinstance(next_watch_list, list) and next_watch_list:
                    for related_data_item in next_watch_list:
                        # Estrai l'ID dal related_data item.
                        # Se next_watch contiene direttamente gli ID:
                        related_id = str(related_data_item) if related_data_item is not None else None
                        # Se next_watch contiene oggetti, es: {'id': 'xyz'}, dovresti estrarre l'id:
                        # related_id = str(related_data_item.get('id')) if isinstance(related_data_item,
```

```
def create_relationship(tx, source_talk_id, related_talk_id):
    """
    Uses MERGE to create a RELATED_TO relationship between two Talk nodes.
    """
    source_id_str = str(source_talk_id)
    related_id_str = str(related_talk_id) # Assicurati che anche related_id sia una stringa

    # Verifica che related_id_str non sia vuoto o problematico
    if not related_id_str: # Questa verifica è già fatta nel loop principale, ma è una doppia sicurezza
        print(f"Warning: Attempted to create relationship from {source_id_str} to an empty related_id. Skipping (inside create_relationship).")
        return

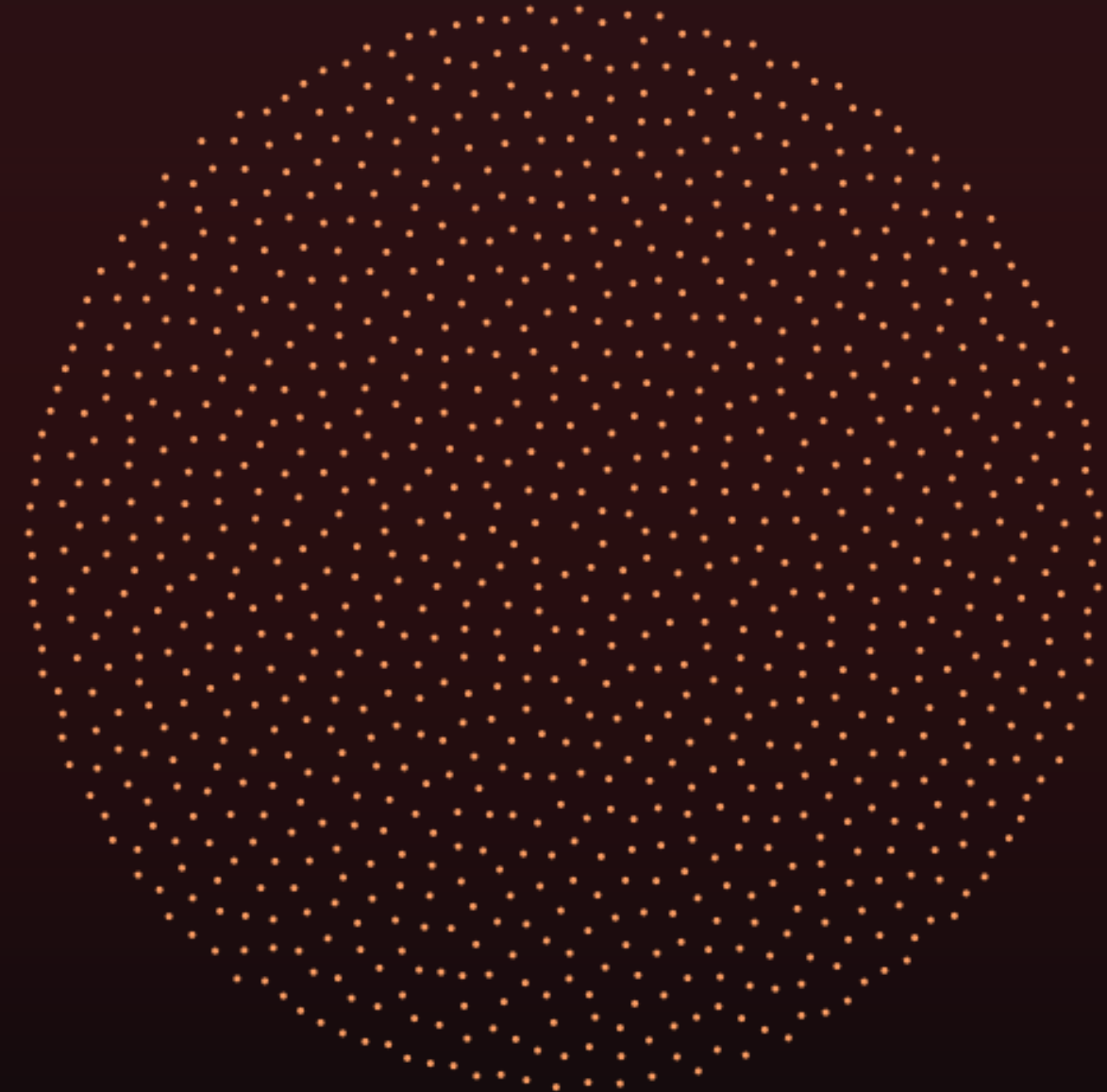
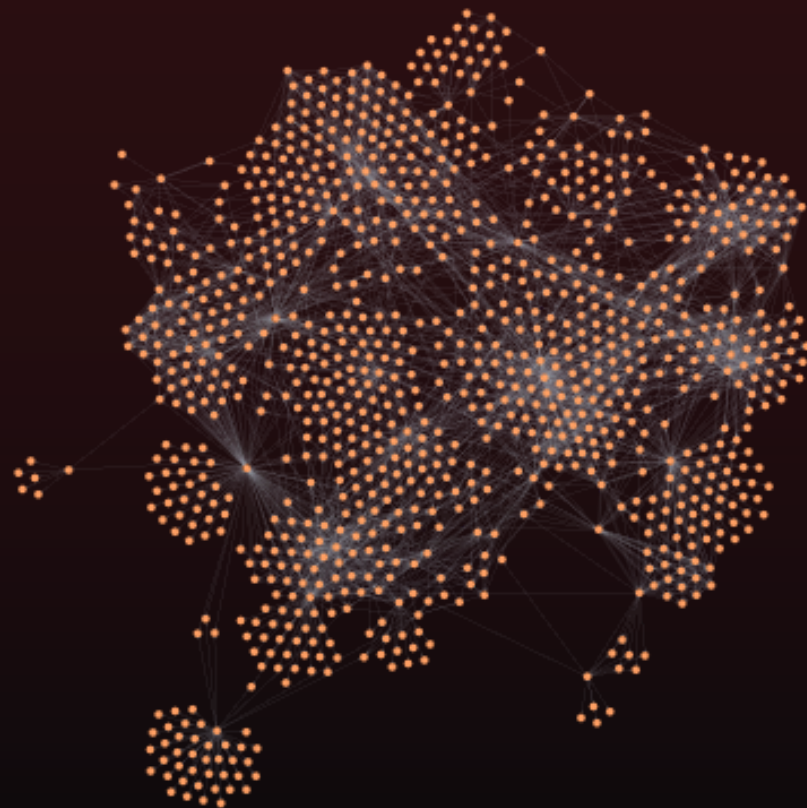
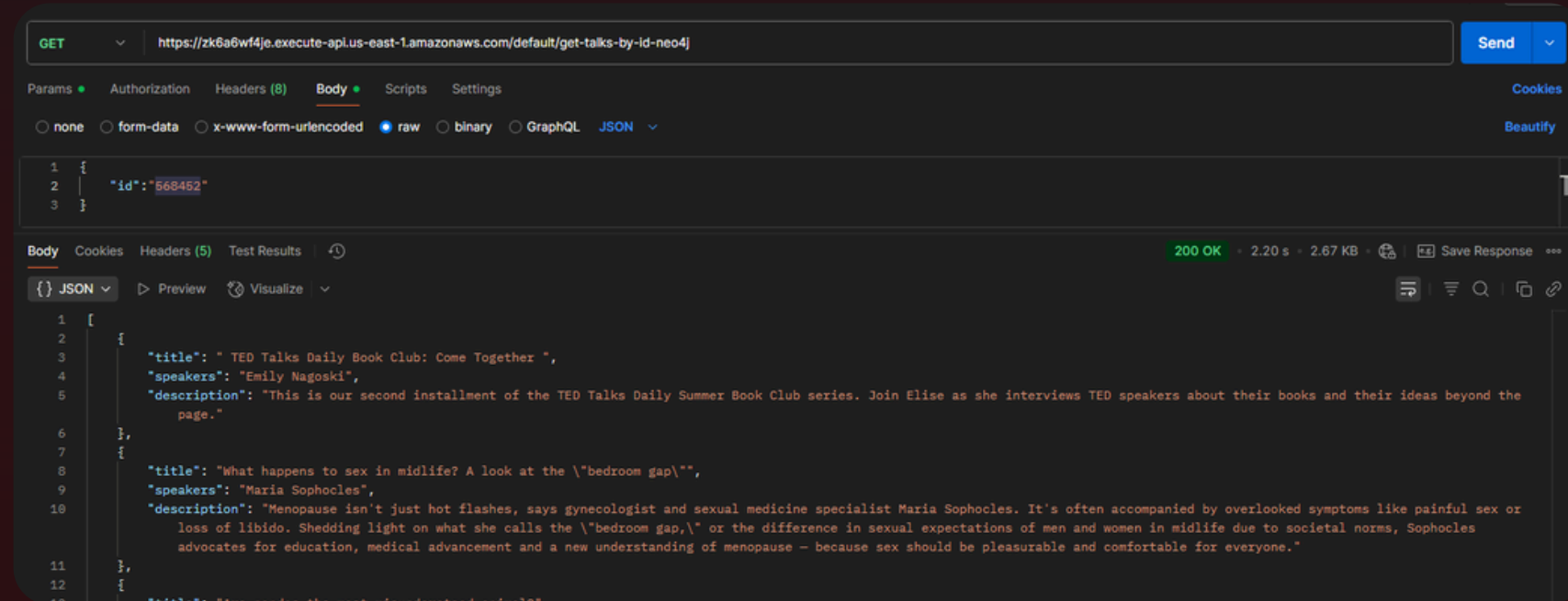
    query = (
        "MATCH (source:Talk {id: $source_id}) "
        "MATCH (related:Talk {id: $related_id}) "
        # Non è necessario "WHERE related.id IS NOT NULL" se $related_id non è mai null o stringa vuota
        # e se c'è un vincolo di esistenza sull'ID o se i nodi sono creati prima.
        # Il MATCH stesso non troverà nodi se l'ID è problematico.
        "MERGE (source)-[:RELATED_TO]->(related)"
    )
    try:
        result = tx.run(query, source_id=source_id_str, related_id=related_id_str)
        summary = result.consume()
        # Logica di conteggio relazioni effettivamente create/unite potrebbe basarsi su summary.counters
    except Exception as e:
        print(f"Error executing relationship MERGE ({source_id_str})->({related_id_str}): {e}. Skipping this relationship.")
        # Non rilanciare l'eccezione per permettere al job di continuare

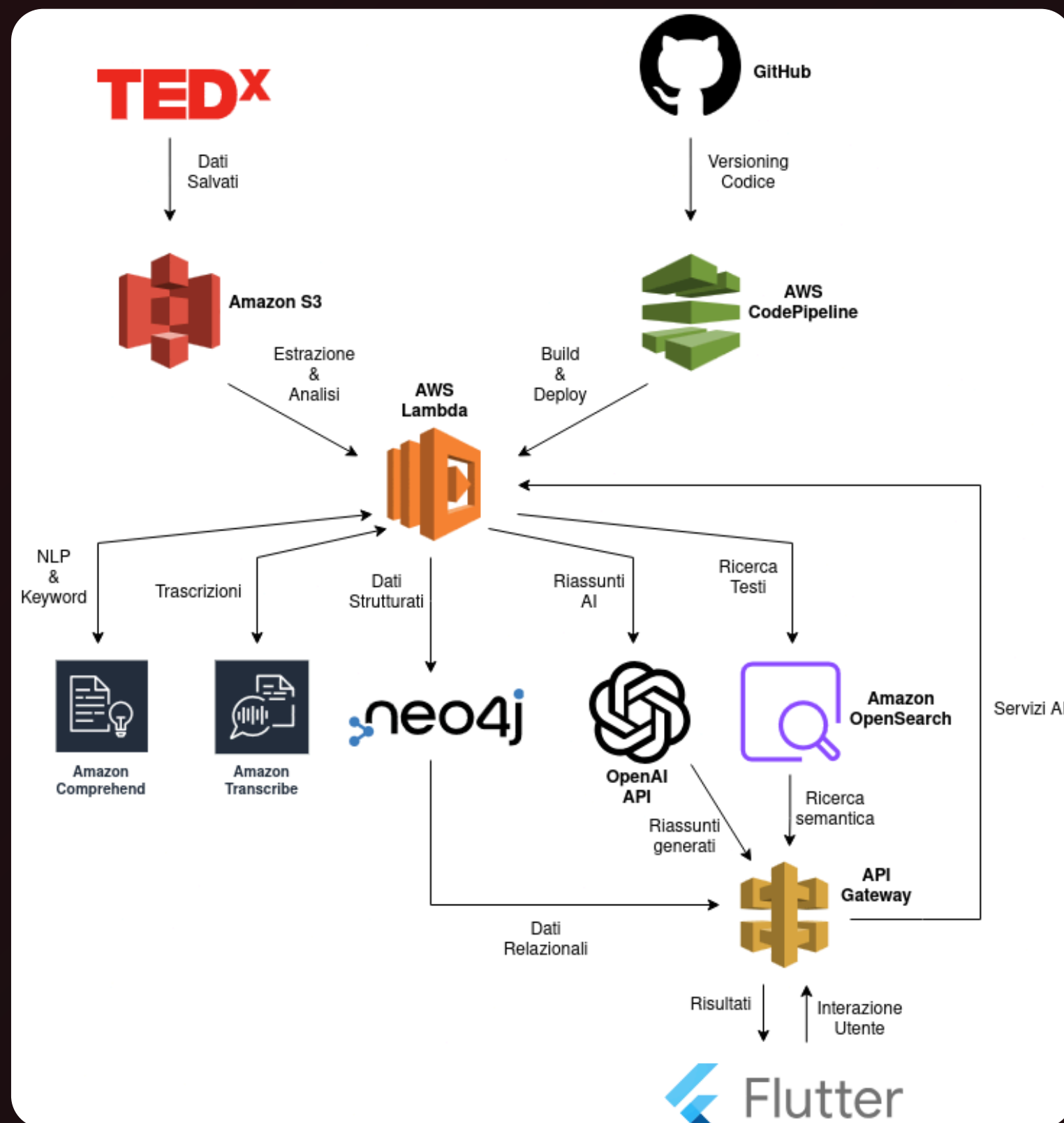
# --- Main Execution Logic for Glue Python Shell ---
if __name__ == "__main__":

    print("Starting AWS Glue Python Shell Job...")
    processed_nodes = 0
    created_relationships_attempts = 0 # Rinominato per chiarezza, dato che MERGE potrebbe non creare
    clients_initialized_successfully = False

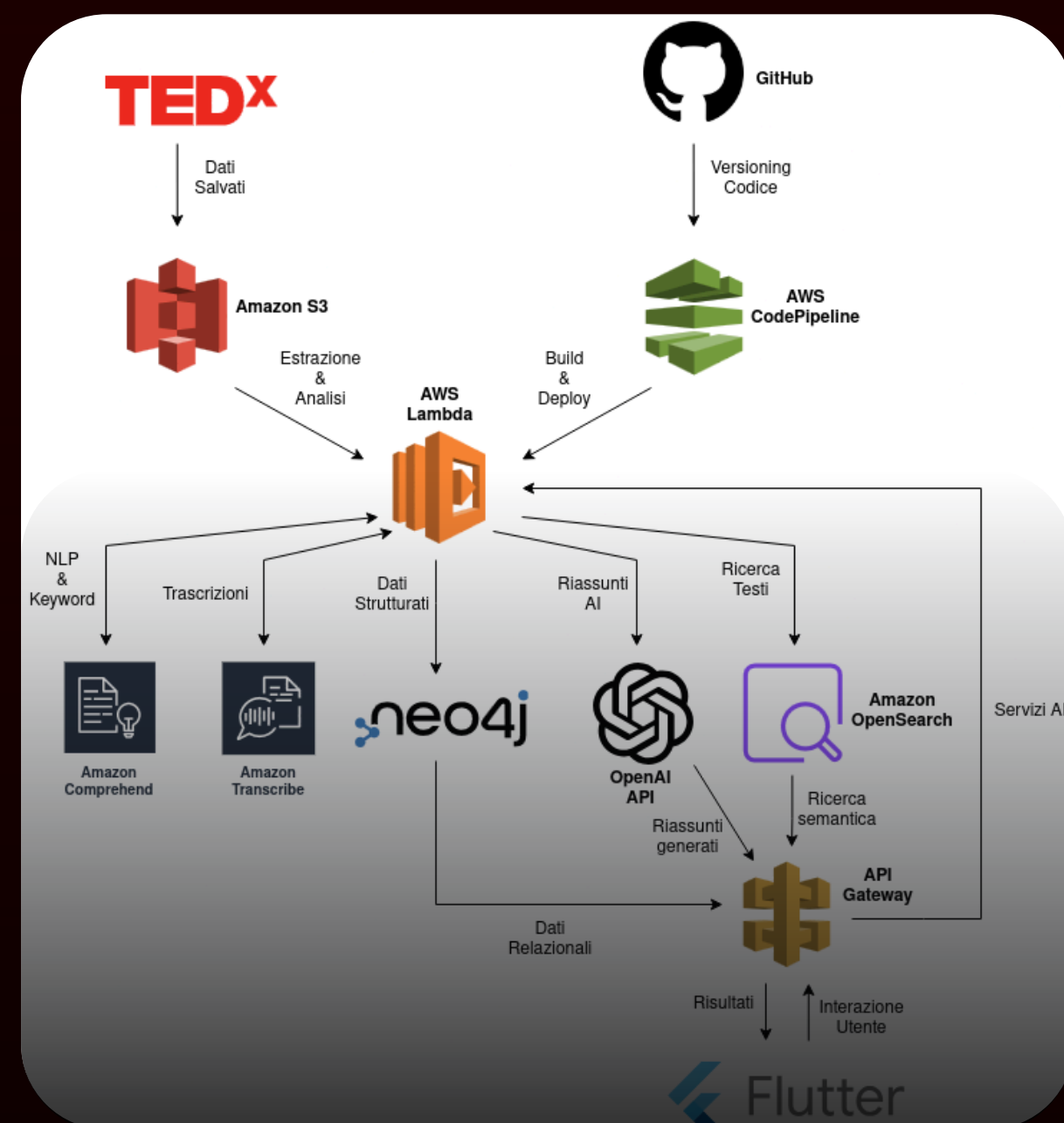
    try:
        print("Attempting to initialize database clients...")
        init_clients()
        clients_initialized_successfully = True
        print("Database clients initialized successfully.")

        db = mongo_client[MONGO_DB_NAME]
        collection = db[MONGO_COLLECTION_NAME]
        print(f"Accessed MongoDB collection: {MONGO_DB_NAME}.{MONGO_COLLECTION_NAME}")
```





MEMO



Pipeline Dati e Elaborazioni AI per TEDx

Sviluppo di una pipeline dati (AWS Glue) che arricchisce i talk TEDx con trascrizioni complete e li carica su MongoDB, affiancata da un servizio API (AWS Lambda) che genera riassunti AI dei contenuti tramite Hugging Face.

tedXjob_V3.py (1/2)

```

# --- INIZIO NUOVA FUNZIONALITÀ: TRASCRIZIONE ---
GRAPHQL_URL = 'https://www.ted.com/graphql'
COMMON_HEADERS = {
    'User-Agent': 'Mozilla/5.0 (X11; Linux x86_64; rv:138.0) Gecko/20100101 Firefox/138.0',
    'Accept': '*/*',
    'client-id': 'Zenith production',
    'content-type': 'application/json',
    'Origin': 'https://www.ted.com',
    'Referer': 'https://www.ted.com/'
}

GRAPHQL_QUERY_TEMPLATE = """
query Transcript($id: ID!, $language: String!) {
  translation(videoId: $id, language: $language) {
    paragraphs {
      cues {
        text
        __typename
      }
      __typename
    }
    __typename
  }
}
"""

def fetch_transcript_for_talk(talk_slug, language="en"): # Default alla lingua inglese
    if not talk_slug:
        return None

    payload = {
        "operationName": "Transcript",
        "variables": {
            "id": talk_slug,
            "language": language
        },
        "query": GRAPHQL_QUERY_TEMPLATE
    }

    headers = COMMON_HEADERS.copy()
    headers['Referer'] = f'https://www.ted.com/talks/{talk_slug}/transcript?language={language}'
    headers['Accept-Language'] = 'en-US,en;q=0.9'

```

tedXjob_V3.py (2/2)

```

try:
    response = requests.post(GRAPHQL_URL, headers=headers, json=payload, timeout=30)
    response.raise_for_status()

    data = response.json()

    if data.get("errors"):
        return None

    translation_block = data.get("data", {}).get("translation")
    if not translation_block:
        return None

    paragraphs = translation_block.get("paragraphs")

    if not paragraphs:
        return None

    transcript_parts = []
    for paragraph in paragraphs:
        if isinstance(paragraph, dict) and paragraph.get("cues"):
            for cue in paragraph.get("cues", []):
                if isinstance(cue, dict):
                    text_content = cue.get("text")
                    if text_content:
                        transcript_parts.append(text_content.strip())

    if not transcript_parts:
        return None

    full_transcript = "\n".join(transcript_parts)
    return full_transcript

except requests.exceptions.RequestException:
    return None
except json.JSONDecodeError:
    return None
except Exception:
    return None

get_transcript_udf = udf(fetch_transcript_for_talk, StringType())
# --- FINE NUOVA FUNZIONALITÀ: TRASCRIZIONE ---

```

```
if not title or not transcript_content:
    missing_fields = []
    if not title: missing_fields.append("'title'")
    if not transcript_content: missing_fields.append("'transcript'") # Modifica: controlla 'transcript'
    error_message = f"Dati { ' e '.join(missing_fields) } mancanti per il talk ID: {talk_id} nel database."
    print(error_message)
    # Logga il documento per aiutare a diagnosticare perché i campi sono mancanti
    print(f"Documento recuperato da MongoDB (ID: {talk_id}): {json.dumps(talk_details, default=str)}")
    return {'statusCode': 400, 'headers': {'**cors_headers', 'Content-Type': 'application/json'}, 'body': json.dumps({'error': error_message})}

print(f"Richiesta di riassunto a Hugging Face per il titolo: '{title}' (transcript preview: '{transcript_content[:100]}...')")
summary_text = get_huggingface_summary(title, transcript_content) # Modifica: chiama nuova funzione

# Controllo robusto del risultato della generazione del riassunto
# I messaggi di errore specifici sono già restituiti da get_huggingface_summary
# Qui si verifica se la stringa restituita indica un errore noto
is_error_string = False
if summary_text:
    summary_lower = summary_text.lower()
    error_keywords = ["errore", "temporaneamente non disponibile", "caricamento", "mancante", "autenticazione", "limite richieste"]
    if any(keyword in summary_lower for keyword in error_keywords):
        is_error_string = True

if not summary_text or is_error_string:
    print(f"Impossibile ottenere il riassunto: {summary_text}")
    status_code = 503 if summary_text and ("caricamento" in summary_text.lower() or "temporaneamente non disponibile" in summary_text.lower()) else 500
    if summary_text and ("configurazione" in summary_text.lower() or "mancante" in summary_text.lower() or "autenticazione" in summary_text.lower()):
        status_code = 500 # Errore di configurazione o autenticazione server-side

    error_body = {'error': f"Impossibile ottenere il riassunto. Dettaglio: {summary_text if summary_text else 'Errore sconosciuto dal servizio di riassunto.'}"
    return {'statusCode': status_code, 'headers': {'**cors_headers', 'Content-Type': 'application/json'}, 'body': json.dumps(error_body)}

# Modifica: aggiorna response_body
response_body = {
    "talk_id_requested": talk_id,
    "mongodb_document_id": talk_details.get("_id"), # _id dovrebbe essere già stringa
    "original_title": title,
    # "original transcript preview": transcript_content[:200] + "..." if transcript_content else None, # Opzionale
    "summary": summary_text # Rinominato da "elaborazione"
}
print(f"Riassunto generato con successo per l'ID: {talk_id}")

return {'statusCode': 200, 'headers': {'**cors_headers', 'Content-Type': 'application/json'}, 'body': json.dumps(response_body, ensure_ascii=False)}
```

```
def lambda_handler(event, context):
    """
    Punto di ingresso della funzione Lambda.
    """
    print(f"Evento ricevuto: {json.dumps(event, default=str)}")
    print(f"Variabili d'ambiente MONGODB_CONN_STRING impostata: {'Si' if MONGODB_CONN_STRING else 'No'}")
    print(f"Variabili d'ambiente HUGGINGFACE_API_TOKEN impostata: {'Si' if HUGGINGFACE_API_TOKEN else 'No'}")
    print(f"Variabili d'ambiente HF_MODEL_ID: {HF_MODEL_ID}")

    # Header CORS per tutte le risposte
    cors_headers = {'Access-Control-Allow-Origin': '*', 'Access-Control-Allow-Headers': 'Content-Type', 'Access-Control-Allow-Methods': 'GET,OPTIONS'}

    # Gestione preflight OPTIONS per CORS
    if event.get('httpMethod') == 'OPTIONS':
        return {'statusCode': 200, 'headers': cors_headers, 'body': ''}

    talk_id = None
    try:
        if isinstance(event.get('queryStringParameters'), dict) and 'id' in event['queryStringParameters']:
            talk_id = event['queryStringParameters']['id']
        elif isinstance(event.get('pathParameters'), dict) and 'id' in event['pathParameters']:
            talk_id = event['pathParameters']['id']
        elif event.get('body'):
            try:
                body = json.loads(event['body'])
                talk_id = body.get('id')
            except json.JSONDecodeError:
                print("Corpo della richiesta JSON non valido.")
                return {'statusCode': 400, 'headers': {'**cors_headers', 'Content-Type': 'application/json'}, 'body': json.dumps({'error': 'Corpo della richiesta JSON non valido'})}
        else:
            talk_id = event.get('id') # Per test console Lambda

    if not talk_id:
        print("Parametro 'id' mancante nella richiesta.")
        return {'statusCode': 400, 'headers': {'**cors_headers', 'Content-Type': 'application/json'}, 'body': json.dumps({'error': "Parametro 'id' mancante nella richiesta."})}

    # Assicura che talk_id sia una stringa, come sembra essere nel DB (_id: "567505")
    talk_id = str(talk_id)

    print(f"Recupero dettagli per l'ID (stringa): {talk_id} da MongoDB.")
    talk_details = get_talk_details_from_mongodb(talk_id)
```

```
    else:
        print(f"Risposta JSON 200 OK inattesa da Hugging Face API per riassunto: {result}")
        return "Risposta inattesa dal servizio di riassunto."
except json.JSONDecodeError as e:
    print(f"Errore nel decodificare la risposta JSON (status 200) da Hugging Face API per riassunto: {e}")
    print(f"Testo della risposta che ha causato l'errore: '{response.text}'")
    return "Errore di comunicazione con il servizio di riassunto (JSON malformato)."
```

```
elif response.status_code == 401:
    print(f"Errore di autenticazione (401) con Hugging Face API (riassunto). Controlla HUGGINGFACE_API_TOKEN. Dettagli: {response.text}")
    return "Errore di autenticazione con il servizio di riassunto."
elif response.status_code == 429:
    print(f"Rate limit superato (429) per Hugging Face API (riassunto). Dettagli: {response.text}")
    return "Limite richieste al servizio di riassunto superato. Riprova più tardi."
elif response.status_code == 503:
    print(f"Modello Hugging Face ({HF_MODEL_ID}) non disponibile o in caricamento (503) (riassunto). Dettagli: {response.text}")
    try:
        error_detail = response.json()
        if isinstance(error_detail, dict) and "error" in error_detail:
            error_msg = error_detail.get("error")
            estimated_time = error_detail.get("estimated_time")
            if "Model" in error_msg and "is currently loading" in error_msg and estimated_time is not None:
                return f"Il modello ({HF_MODEL_ID}) per il riassunto è in fase di caricamento (stimato: {estimated_time}s). Riprova tra poco."
            return f"Servizio di riassunto temporaneamente non disponibile: {error_msg}"
    except json.JSONDecodeError:
        return f"Servizio di riassunto temporaneamente non disponibile (503). Dettagli: {response.text[:200]}". # Mostra parte del testo se non è 350"
```



```
payload = {
    "inputs": prompt_content,
    "parameters": {
        "max_new_tokens": 350,      # Adattato per un riassunto (es. ~250-500 parole)
        "temperature": 0.6,        # Leggermente più basso per riassunti fattuali
        "return_full_text": False,  # Solo il testo generato
    },
    "options": {
        "use_cache": True,
        "wait_for_model": True
    }
}

print(f"Invio richiesta a Hugging Face API: {api_url} per il modello {HF_MODEL_ID}")
transcript_preview = transcript_content[:100] + "..." if len(transcript_content) > 100 else transcript_content
print(f"Payload (title: '{title}', transcript_preview: '{transcript_preview}', ...)")

try:
    response = requests.post(api_url, headers=headers, json=payload, timeout=55) # Timeout per Lambda

    print(f"Hugging Face API Response Status Code: {response.status_code}")
    # print(f"Hugging Face API Response Headers: {response.headers}") # Può essere verboso

    response_text_preview = response.text[:500] if response.text else "VUOTO"
    print(f"Hugging Face API Response Text (preview): '{response_text_preview}'")

    if response.status_code == 200:
        try:
            result = response.json()
            if isinstance(result, list) and len(result) > 0 and "generated_text" in result[0]:
                return result[0]["generated_text"].strip()
            elif isinstance(result, dict) and "generated_text" in result: # Alcuni modelli restituiscono un dict
                return result["generated_text"].strip()
            elif isinstance(result, dict) and "error" in result:
                error_msg = result.get("error")
                estimated_time = result.get("estimated_time")
                warnings = result.get("warnings")
                log_msg = f"Hugging Face API ha restituito 200 OK ma con errore JSON (riassunto): {error_msg}"
                if warnings: log_msg += f" Warnings: {warnings}"
                print(log_msg)
                if "Model" in error_msg and "is currently loading" in error_msg and estimated_time is not None:
                    return f"Il modello ({HF_MODEL_ID}) è in fase di caricamento per il riassunto (stimato: {estimated_time}s). Riprova tra poco."
        except json.JSONDecodeError:
            return f"Servizio di riassunto temporaneamente non disponibile (503). Dettagli: {response.text[:200]}" # Mostra parte del testo se non è JSON
        else:
            print(f"Errore HTTP {response.status_code} da Hugging Face API (riassunto). Dettagli: {response.text}")
            return f"Errore {response.status_code} dal servizio di riassunto. Dettagli: {response.text[:200]}"

except requests.exceptions.Timeout:
    print("Timeout durante la chiamata a Hugging Face API per riassunto.")
    return "Il servizio di riassunto ha impiegato troppo tempo a rispondere."
except requests.exceptions.RequestException as req_err:
    print(f"Errore di richiesta generico con Hugging Face API per riassunto: {req_err}")
    return "Errore di connessione con il servizio di riassunto."
except Exception as e:
    print(f"Errore generico non gestito durante la generazione del riassunto con Hugging Face: {e}")
    import traceback
    traceback.print_exc()
    return "Errore imprevisto durante la generazione del riassunto."
```

```
# Basandoci sull'immagine_id: "567505", l'ID è una stringa.
# Se talk_id_str fosse un ObjectId valido, la query sarebbe ObjectId(talk_id_str)
query = {"_id": talk_id_str}

print(f"Esecuzione query su MongoDB: {query} nella collezione {MONGODB_COLLECTION_NAME}")
document = collection.find_one(query)

if document:
    print(f"Documento trovato in MongoDB per l'ID {talk_id_str}")
    # Se _id fosse un ObjectId, convertirlo in stringa per la serializzazione JSON
    if '_id' in document and isinstance(document['_id'], ObjectId):
        document['_id'] = str(document['_id'])
    return document
else:
    print(f"Nessun documento trovato in MongoDB per l'ID: {talk_id_str} con la query {query}")
    return None

except Exception as e: # Potrebbe includere bson.errors.InvalidId se si tentasse di convertire una stringa non valida in ObjectId
    print(f"Errore durante l'accesso a MongoDB o ID non valido: {e}")
    return None

# Modificata: nome, parametri, prompt, parametri modello
def get_huggingface_summary(title, transcript_content):
    """
    Chiama le API di Hugging Face Inference per ottenere un riassunto del transcript.
    """
    if not HUGGINGFACE_API_TOKEN:
        print("Errore: HUGGINGFACE_API_TOKEN non impostato.")
        return "Configurazione API mancante."
    if not HF_MODEL_ID:
        print("Errore: HF_MODEL_ID non impostato.")
        return "Configurazione modello mancante."

    api_url = f"https://api-inference.huggingface.co/models/{HF_MODEL_ID}"
    headers = {"Authorization": f"Bearer {HUGGINGFACE_API_TOKEN}"}

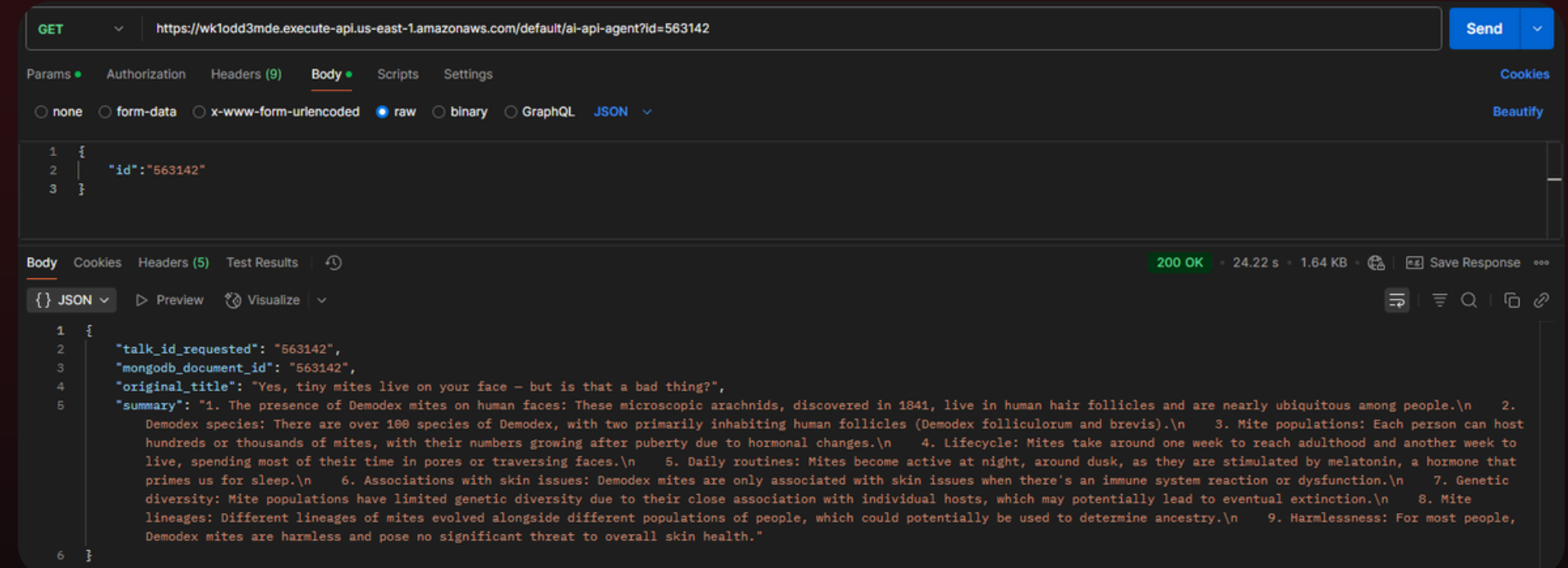
    # Nuovo prompt per il riassunto in inglese, basato sul transcript
    prompt_content = f"""
    [INST] You are an expert assistant skilled in summarizing talk content.
    Talk Title: "{title}"
    Talk Transcript: "{transcript_content}"
    """
```

```
except json.JSONDecodeError:
    return f"Servizio di riassunto temporaneamente non disponibile (503). Dettagli: {response.text[:200]}" # Mostra parte del testo se non è JSON
    return "Servizio di riassunto temporaneamente non disponibile (503)." # Fallback
else:
    print(f"Errore HTTP {response.status_code} da Hugging Face API (riassunto). Dettagli: {response.text}")
    return f"Errore {response.status_code} dal servizio di riassunto. Dettagli: {response.text[:200]}"

except requests.exceptions.Timeout:
    print("Timeout durante la chiamata a Hugging Face API per riassunto.")
    return "Il servizio di riassunto ha impiegato troppo tempo a rispondere."
except requests.exceptions.RequestException as req_err:
    print(f"Errore di richiesta generico con Hugging Face API per riassunto: {req_err}")
    return "Errore di connessione con il servizio di riassunto."
except Exception as e:
    print(f"Errore generico non gestito durante la generazione del riassunto con Hugging Face: {e}")
    import traceback
    traceback.print_exc()
    return "Errore imprevisto durante la generazione del riassunto."
```

Risultato

```
_id: "567505"
slug: "ben_proudfoot_the_true_story_of_the_iconic_tagline_because_i_m_worth_i..."
speakers: "Ben Proudfoot"
title: "The true story of the iconic tagline “Because I’m worth it.” | The Fin..."
url: "https://www.ted.com/talks/ben_proudfoot_the_true_story_of_the_iconic_t..."
transcript: "- I don't remember exactly, but ...
              I use the most expensive
              hair colo..."
description: "From two-time Oscar winner Ben Proudfoot comes THE FINAL COPY OF ILON ..."
duration: "1059"
publishedAt: "2025-03-07T13:49:56Z"
tags: Array (5)
next_watch: Array (5)
```



Extra in vista per Flutter



Obiettivo:

Per dare vita alla futura WebApp interattiva e permettere l'esplorazione efficace delle mappe mentali, è stata implementata una funzione AWS Lambda scritta in Python, progettata per interagire con il nostro database a grafo Neo4j.

2. Restituisce i talk trovati per aggiornare mappa e risultati in tempo reale.

1. Riceve la stringa di ricerca.

```
variabili d'ambiente (da configurare nella Lambda)
NEO4J_URI = os.environ.get('NEO4J_URI')
NEO4J_USER = os.environ.get('NEO4J_USER')
NEO4J_PASSWORD = os.environ.get('NEO4J_PASSWORD')

driver = None

def get_neo4j_driver():
    global driver
    if driver is None:
        try:
            driver = GraphDatabase.driver(NEO4J_URI, auth=basic_auth(NEO4J_USER, NEO4J_PASSWORD))
            # Verifica la connessione (opzionale ma utile per il debug iniziale)
            driver.verify_connectivity()
            print("Successfully connected to Neo4j.")
        except Exception as e:
            print(f"Error connecting to Neo4j: {e}")
            # Se la connessione fallisce, driver rimarrà None e gestito dopo
            # Potresti voler sollevare un'eccezione qui se la connessione è critica all'avvio
            raise ConnectionError(f"Failed to connect to Neo4j: {e}") from e
    return driver

def get_connected_nodes(tx, node_id_param):
    query = (
        "MATCH (startNode {id: $node_id_param})--(connectedNode) "
        "RETURN connectedNode.title AS title, "
        "        connectedNode.speakers AS speakers, "
        "        connectedNode.description AS description"
    )
    result = tx.run(query, node_id_param=node_id_param)

    nodes_data = []
    for record in result:
        nodes_data.append({
            "title": record["title"],
            "speakers": record["speakers"], # Assumendo sia una lista o una stringa
            "description": record["description"]
        })
    return nodes_data
```

```
print(f"Found {len(found_nodes_list)} matching nodes.")

return {
    'statusCode': 200,
    'headers': {
        'Content-Type': 'application/json',
        'Access-Control-Allow-Origin': '*'
    },
    'body': json.dumps(found_nodes_list)
}

except ConnectionError as ce:
    print(f"Connection Error: {ce}")
    return {
        'statusCode': 503,
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'error': 'Database connection failed', 'details': str(ce)})
    }

except Exception as e:
    print(f"Error processing request: {e}")
    return {
        'statusCode': 500,
        'headers': {'Content-Type': 'application/json'},
        'body': json.dumps({'error': 'Internal server error', 'details': str(e)})
    }
```

```
variabili d'ambiente (da configurare nella Lambda)
NEO4J_URI = os.environ.get('NEO4J_URI')
NEO4J_USER = os.environ.get('NEO4J_USER')
NEO4J_PASSWORD = os.environ.get('NEO4J_PASSWORD')

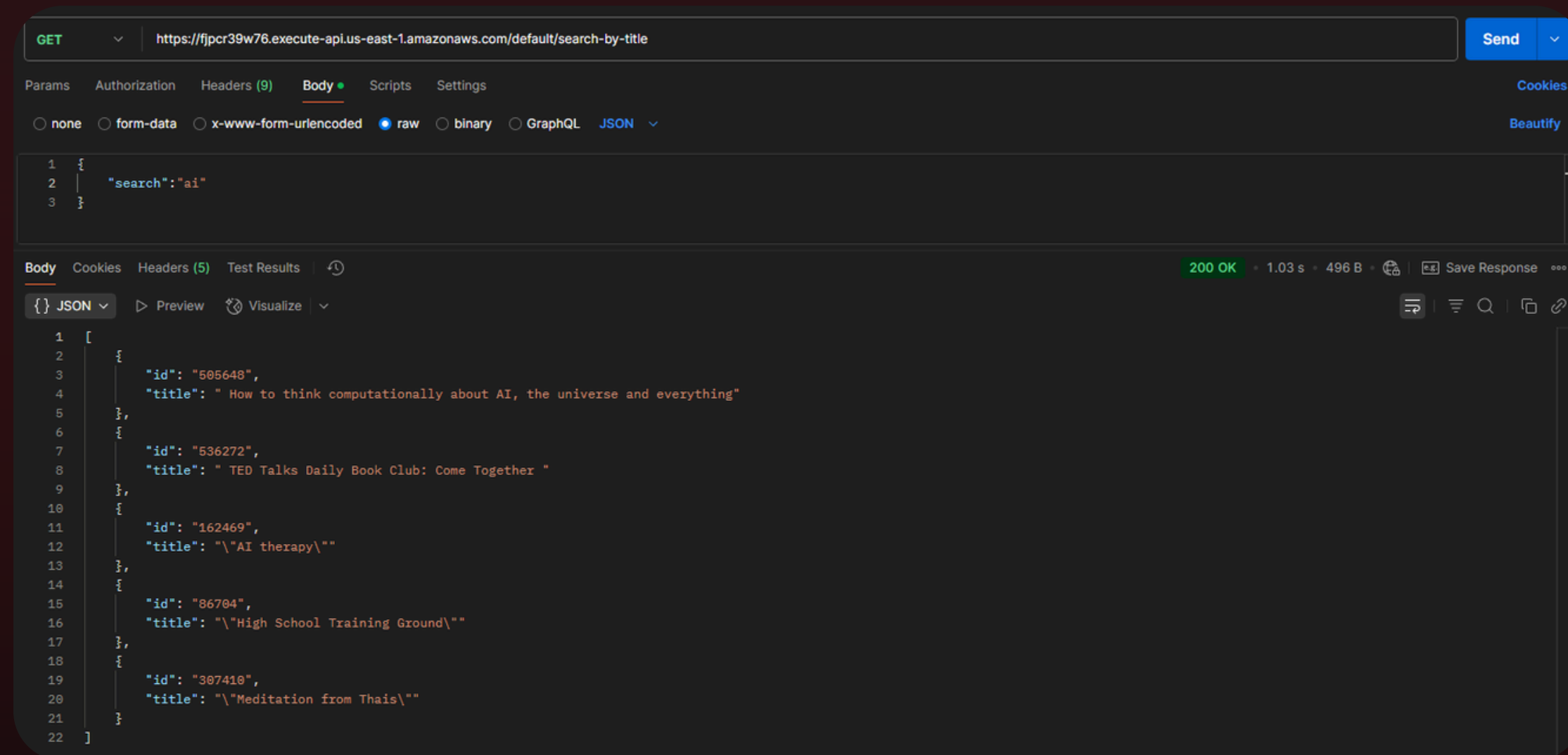
driver = None

def get_neo4j_driver():
    global driver
    if driver is None:
        try:
            driver = GraphDatabase.driver(NEO4J_URI, auth=basic_auth(NEO4J_USER, NEO4J_PASSWORD))
            # Verifica la connessione (opzionale ma utile per il debug iniziale)
            driver.verify_connectivity()
            print("Successfully connected to Neo4j.")
        except Exception as e:
            print(f"Error connecting to Neo4j: {e}")
            # Se la connessione fallisce, driver rimarrà None e gestito dopo
            # Potresti voler sollevare un'eccezione qui se la connessione è critica all'avvio
            raise ConnectionError(f"Failed to connect to Neo4j: {e}") from e
    return driver

def get_connected_nodes(tx, node_id_param):
    query = (
        "MATCH (startNode {id: $node_id_param})--(connectedNode) "
        "RETURN connectedNode.title AS title, "
        "        connectedNode.speakers AS speakers, "
        "        connectedNode.description AS description"
    )
    result = tx.run(query, node_id_param=node_id_param)

    nodes_data = []
    for record in result:
        nodes_data.append({
            "title": record["title"],
            "speakers": record["speakers"], # Assumendo sia una lista o una stringa
            "description": record["description"]
        })
    return nodes_data
```

2. Esegue query Cypher su Neo4j cercando talk i cui titoli contengono l'input utente.



Criticità



01. Scraping delle trascrizioni

Inizialmente, il tentativo di estrarre le trascrizioni dei talk da tedx.com tramite scraping dell'HTML ha presentato significative difficoltà operative. Successivamente, su indicazione, abbiamo implementato con successo il recupero delle trascrizioni formulando richieste mirate all'API GraphQL di TED.com



02. Tempi lunghi di esecuzione

L'elaborazione del modello a grafo tramite script Python su EC2 si è rivelata un processo dispendioso in termini di tempo, con durate di diverse ore. Lo script lanciato in AWS Glue, pur mostrando un'esecuzione più lenta del previsto, ha comunque operato entro tempi accettabili (decine di minuti), commisurati al carico di lavoro.



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Possibili Evoluzioni

Capitalizzando sulle attuali implementazioni e analizzando le performance, le prossime evoluzioni mirano a ottimizzare, scalare e arricchire ulteriormente il sistema.

Ottimizzazione Pipeline Dati e Generazione Grafo

Riduzione drastica tempi generazione grafo (EC2) e affinamento script Glue. Esplorazione parallelizzazione, algoritmi e infrastrutture ottimizzate per Neo4j.

Nuove features AI

Ampliamento capacità AI (Q&A, topic modeling). Standardizzazione approccio GraphQL per integrare nuove fonti dati esterne in modo efficiente.

Monitoraggio delle performance

Monitoraggio end-to-end di performance e costi. Strategie di scaling automatico per gestire crescita dati e picchi di carico.



UNIVERSITÀ
DEGLI STUDI
DI BERGAMO

Team TEDxGRAPH



**Davide
Bonsembiante**

Mat. 1086863



**Thomas
Paganelli**

Mat. 1085805



**Lorenzo
Gallizioli**

Mat. 1087404